Incident Tracker

An incident tracker for on-call teams: open an incident with a severity and a service, follow it through investigating to resolved, and keep a timeline of notes. FastAPI, PostgreSQL and React, shipped through tests, security scans, GHCR, Helm, Argo CD and Terraform on AWS.

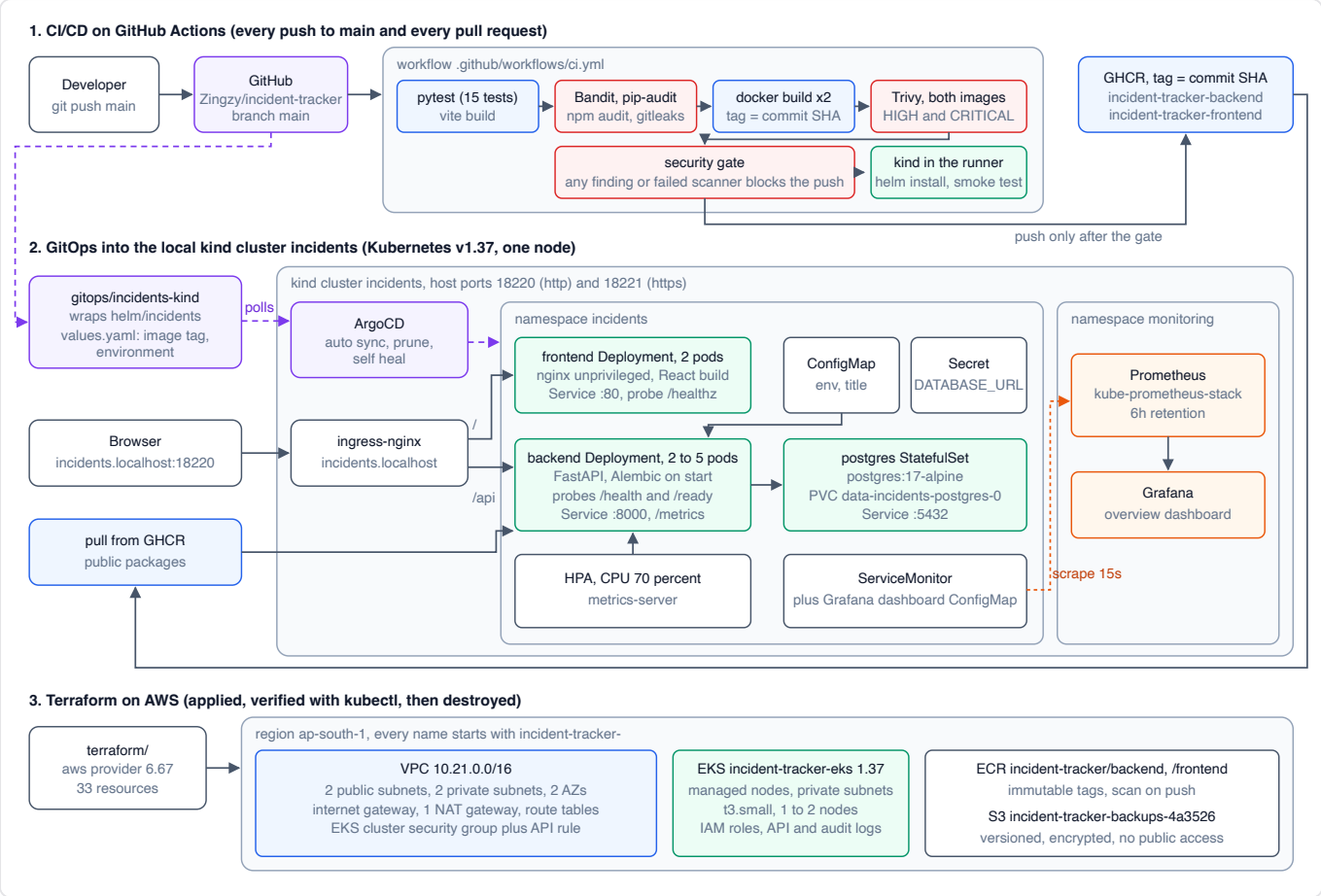
STUDENT	Aditya Singhi
REPOSITORY	github.com/Zingzy/incident-tracker
GREEN PIPELINE	actions/runs/38911274910
IMAGES	ghcr.io/zingzy/incident-tracker-backend ghcr.io/zingzy/incident-tracker-frontend
AWS REGION	ap-south-1 (Mumbai), destroyed after capture

MODULE	TOPIC	POINTS
M1	Application, frontend, backend and database	10
M2	Testing, pytest	10
M3	Git and GitHub	5
M4	Docker and Compose	10
M5	CI/CD, GitHub Actions	15
M6	DevSecOps, Trivy	5
M7	Terraform on AWS	15
M8	Kubernetes and Helm	15
M9	Prometheus and Grafana	10
M10	README and live demo	5

Architecture

`overview`

Every push to `main` runs the pipeline: tests and the frontend build, then SAST, dependency, secret and image scans behind one security gate. Only images that passed are pushed to GHCR, tagged with the commit SHA, and installed with Helm on a kind cluster inside the runner. Argo CD deploys the promoted SHA to the local kind cluster, and Terraform provisions the VPC and EKS cluster on AWS.



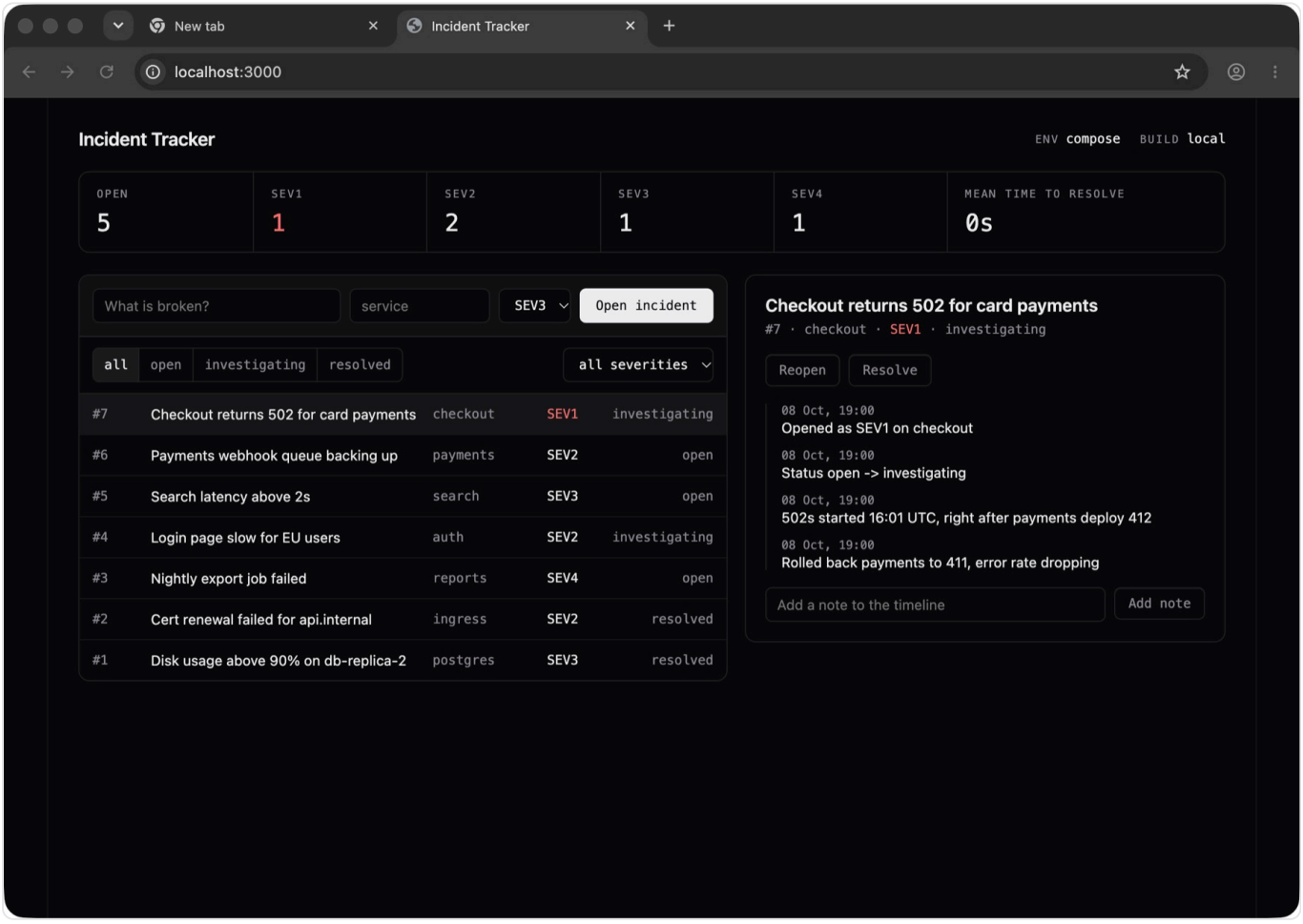
`docs/architecture.svg`

M1 · Application

10 pts

FastAPI with SQLAlchemy and PostgreSQL. Incidents carry a severity (SEV1 to SEV4), a service and a status, and every status change writes a timeline note automatically.

CRITERION	PTS	EVIDENCE
FastAPI starts and responds to `/health`	2	`GET /health` returns `{"status":"UP"}`, also the Kubernetes liveness probe. `/ready` checks the database.
At least 4 REST endpoints, GET, POST, PUT, DELETE	3	`GET/POST /api/incidents`, `GET/PUT/DELETE /api/incidents/{id}`, `POST /api/incidents/{id}/status`, `GET/POST /api/incidents/{id}/notes`, `GET /api/stats`, `GET /api/info`
PostgreSQL table managed by Alembic	2	`backend/alembic/versions/0001_create_incidents.py` creates `incidents` and `notes`. Replicas take a Postgres advisory lock so only one runs the migration.
Frontend renders and calls the API	2	React and Vite in `frontend/`. nginx proxies `/api` to the backend.
Responsive, usable UI	1	Stats row, filters, incident list, timeline with notes, resolve and reopen.



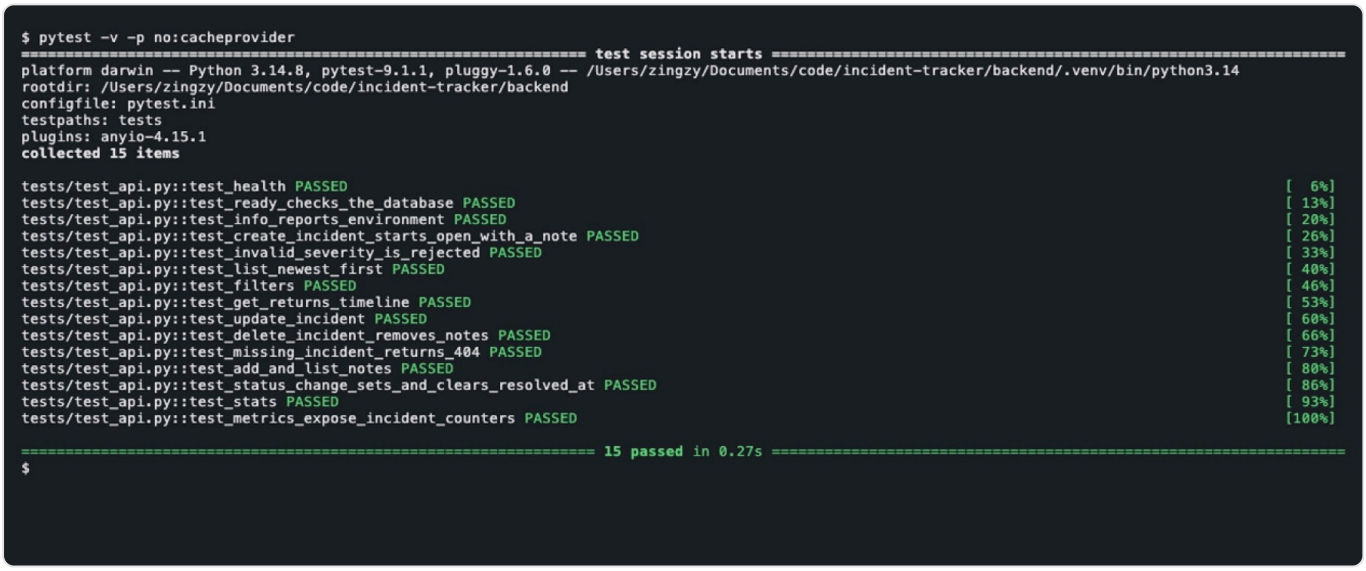
11 · the app at localhost:3000, running under docker compose

M2 · Testing

10 pts

15 tests over every endpoint, the filters, the timeline, resolving and reopening, the stats, the metrics, and the error paths (422 for an unknown severity or status, 404 for a missing incident).

CRITERION	PTS	EVIDENCE
`pytest` runs without errors	3	15 passed, locally and in the `backend-test` CI job.
At least 5 tests over at least 3 endpoints	4	15 tests in `backend/tests/test_api.py`, covering `/health`, `/ready`, every `/api` route and `/metrics`.
Test database, not production	2	`conftest.py` overrides `get_db` with an in-memory SQLite database, created and dropped per test.
`pytest.ini` or `conftest.py`	1	Both: `backend/pytest.ini` and `backend/tests/conftest.py`.

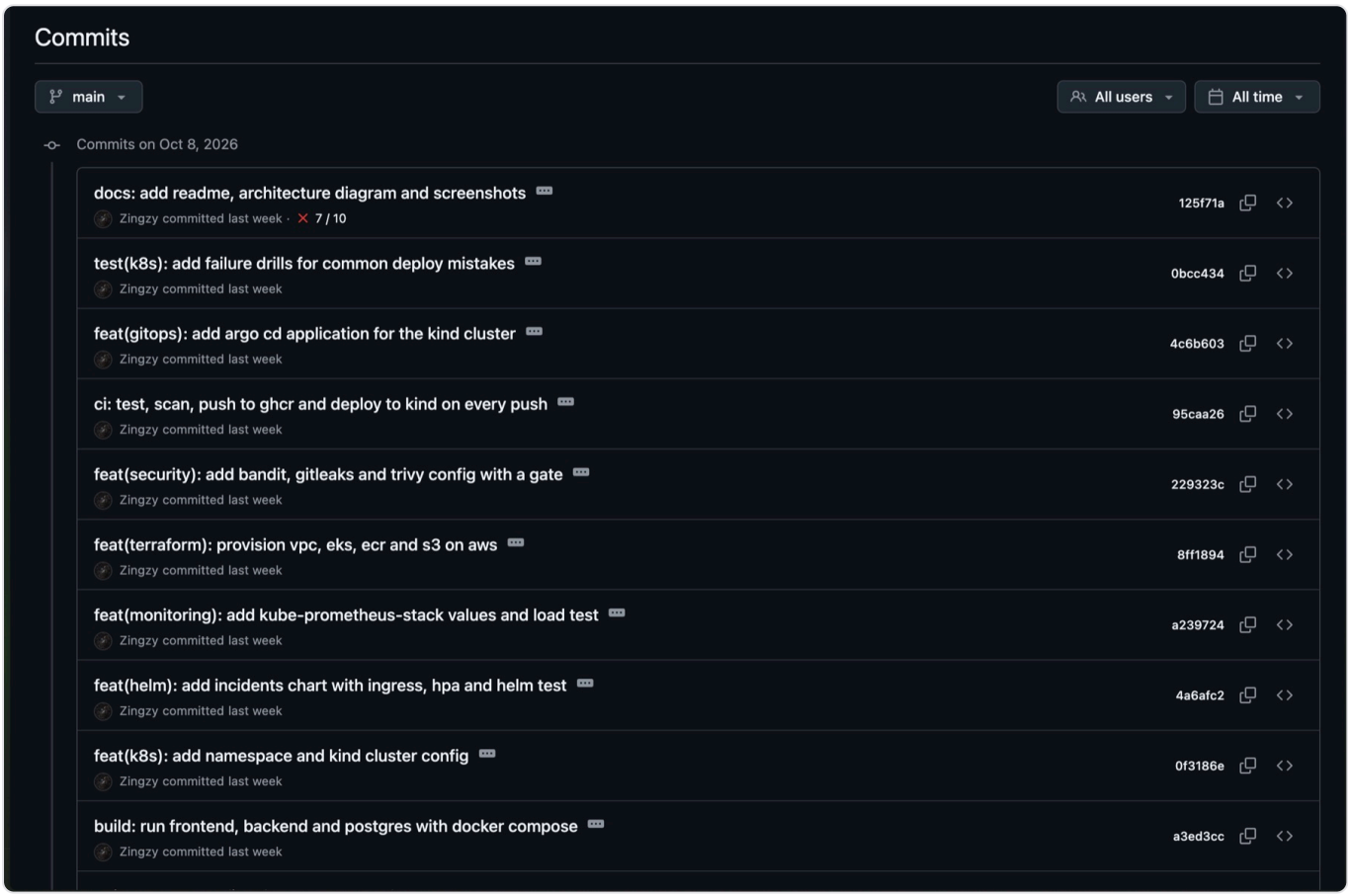


12 · pytest -v, 15 passed

M3 · Git and GitHub

5 pts

CRITERION	PTS	EVIDENCE
Public repository	2	github.com/Zingzy/incident-tracker, public.
Meaningful commit messages	2	21 conventional commits, each with a body explaining why, for example `feat(helm): add incidents chart with ingress, hpa and helm test`.
`.gitignore` covers `.env`, `__pycache__`, `node_modules`, `.venv`	1	All four, plus Terraform state, plans, `terraform.tfvars` and scanner reports.

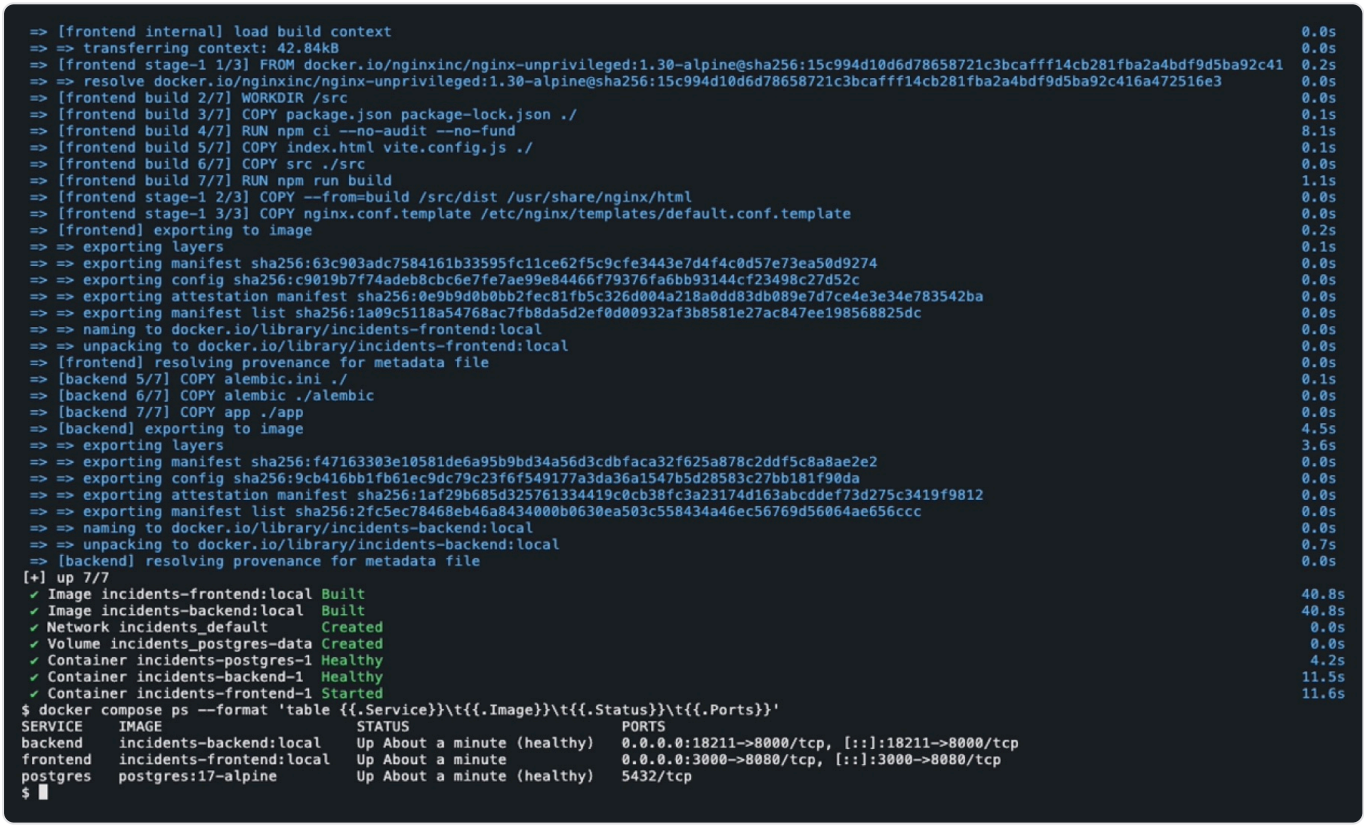


15 · commit history on GitHub

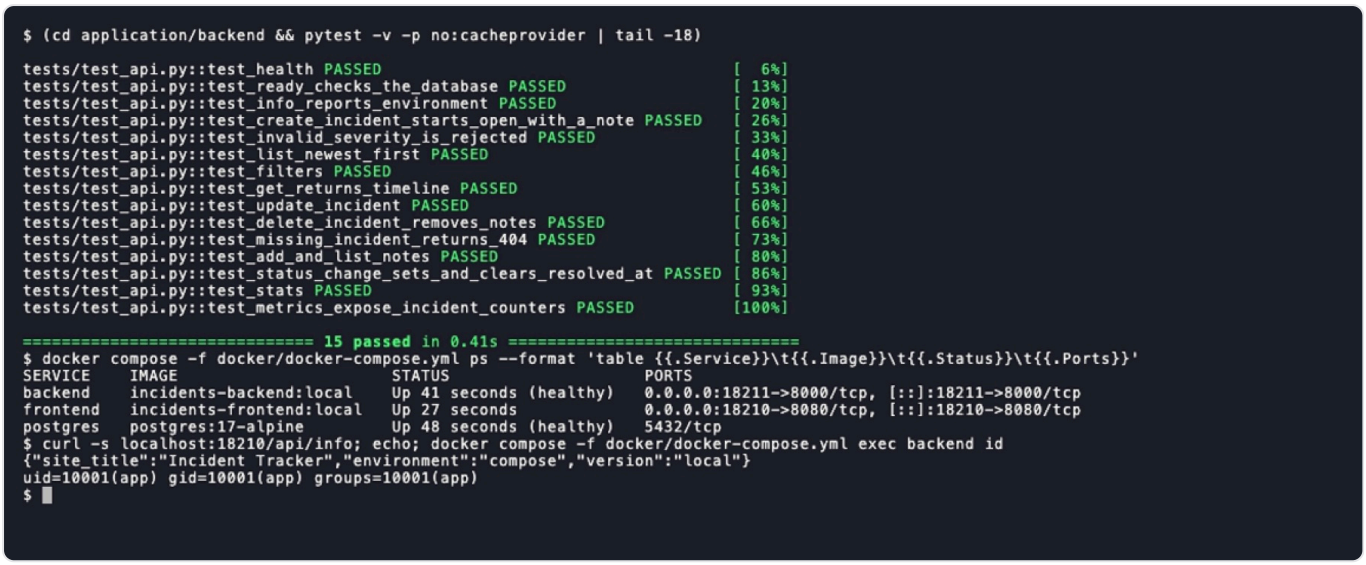
M4 · Docker

10 pts

CRITERION	PTS	EVIDENCE
`backend/Dockerfile` builds	2	`python:3.12-alpine`, pip removed after install, runs `alembic upgrade head` then uvicorn.
`frontend/Dockerfile` is multi-stage	3	`node:22-alpine` builds `dist/`, `nginxinc/nginx-unprivileged:1.30-alpine` serves it.
Images run as non-root	2	Backend uid 10001, frontend uid 101. Both checked with `id` inside the running containers.
`docker compose up --build` starts all three	3	postgres, backend and frontend come up healthy. The frontend waits for the backend's `/ready` check, which waits for Postgres.



10 · docker compose up --build from the repo root



02 · both containers running as non-root users

M5 · CI/CD

15 pts

`.github/workflows/ci.yml` runs on every push to `main`. Both images are built once and handed between jobs as a tar, so the image pushed is the exact image Trivy scanned.

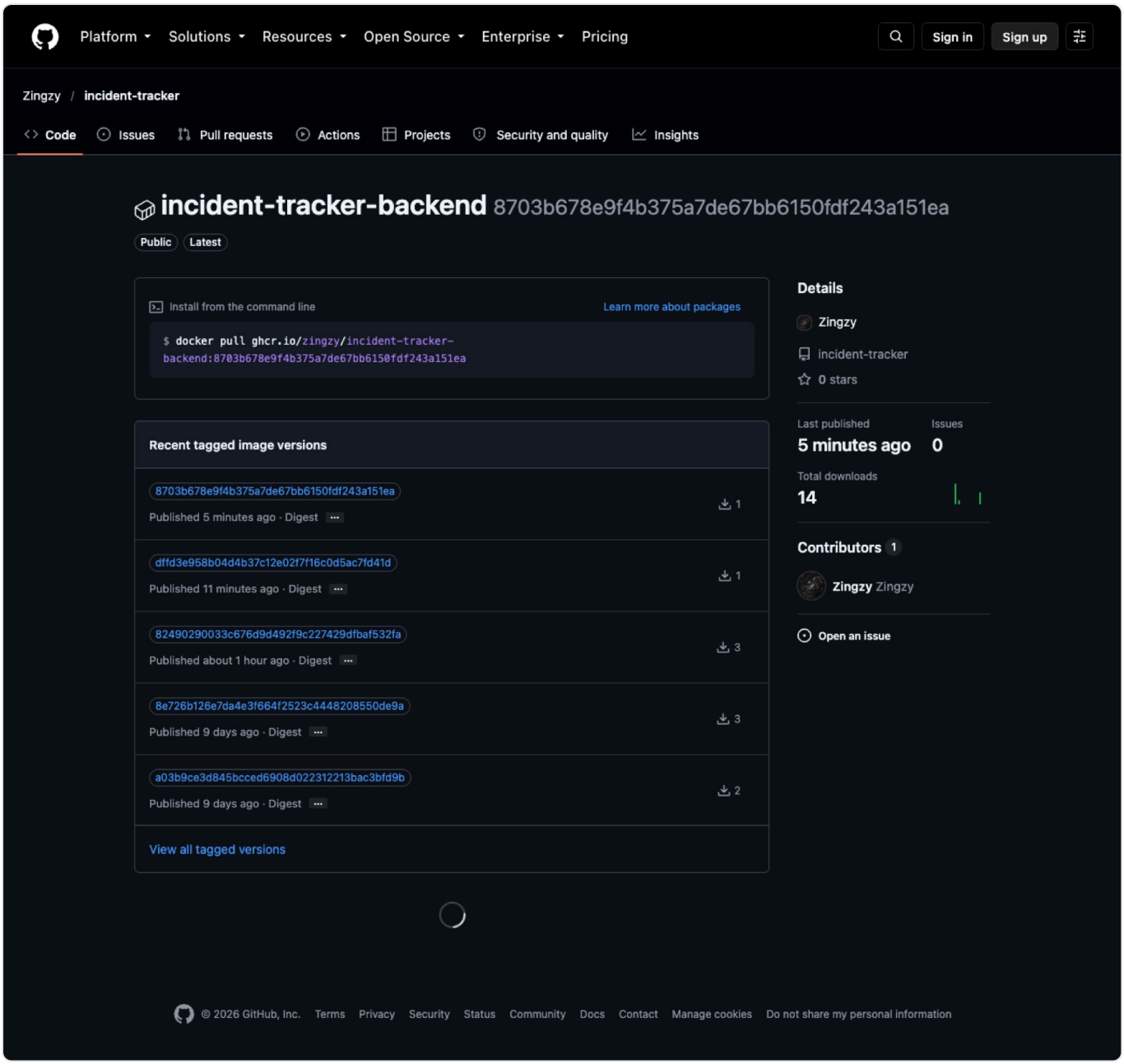
```
backend tests + frontend build
  -> Bandit, pip-audit + npm audit, gitleaks
  -> docker build (SHA tag) -> Trivy -> security gate
  -> push to GHCR -> kind in the runner, helm install, smoke test
```

CRITERION	PTS	EVIDENCE
Workflow in `.github/workflows/`	1	`ci.yml`, 10 jobs.
Triggers on push to `main`	1	`on: push: branches: [main]`, plus pull requests and manual runs.
pytest job fails the build	3	`backend-test` runs `pytest -v`. Every later job depends on it.
Frontend built in the pipeline	2	`frontend-build` runs `npm ci` and `npm run build`.
Images built for frontend and backend	3	`docker-build` builds both from their Dockerfiles.
Images pushed to GHCR	3	`push` job, only after the security gate passes.
Tags use the commit SHA	2	Tagged `${{ github.sha }}`, never `latest`. The smoke test checks `/api/info` reports that exact SHA.

Green run: actions/runs/38911274910 on `8249029`, all 10 jobs passed.



07 · tests, scans, the gate, the GHCR push and the in-runner deploy from run 38911274910



06 · GHCR package with commit SHA tags

M6 · DevSecOps

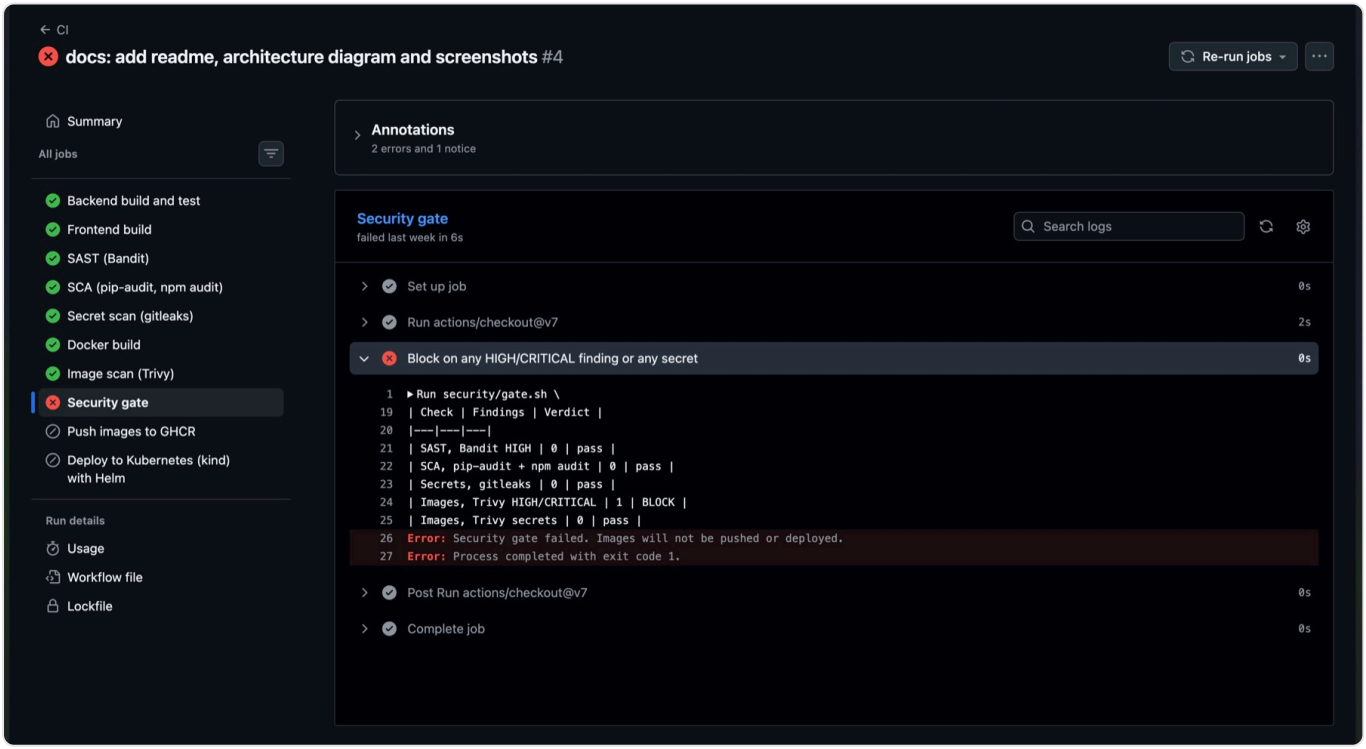
5 pts

CRITERION	PTS	EVIDENCE
Trivy scans both images in CI	3	`image-scan` job runs `trivy image` on the backend and frontend images built for the commit.
Pipeline fails on HIGH or CRITICAL	1	`security/trivy.yaml` limits findings to HIGH and CRITICAL. `security/gate.sh` fails the build on any finding, any secret, or a scanner job that did not finish. Push and deploy depend on the gate.
Explain one CVE	1	Below.

WHAT TRIVY FOUND

Trivy scanned the Alpine OS packages in both images and the Python packages in the backend image for known vulnerabilities, and both images for leaked secrets. On run 38091482670 it found CVE-2026-4775, a HIGH severity bug in libtiff (`tiff 4.7.1-r0`) where a crafted TIFF file can cause code execution or a crash. It came with the `nginx-unprivileged:1.30-alpine` base of the frontend image, not our code, and Alpine had already shipped the fix in `4.7.2-r0`.

The gate blocked the push and the deploy. The frontend Dockerfile now runs `apk upgrade` in the runtime stage and switches back to uid 101, and the next run scanned clean: 0 HIGH or CRITICAL findings and 0 secrets in both images.



16 · the security gate blocking the push on the Trivy finding

M7 · Terraform on AWS

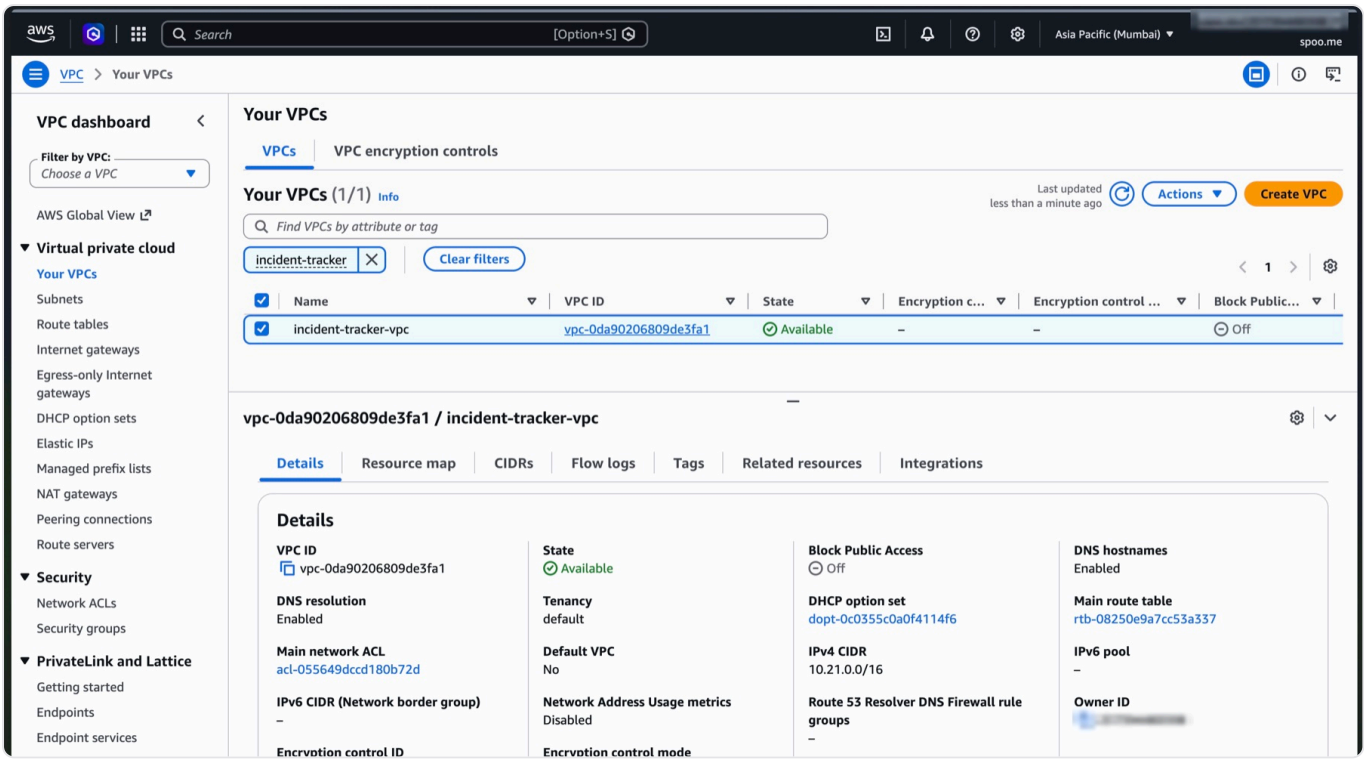
15 pts

A VPC `10.21.0.0/16` with two public and two private subnets across two AZs, an internet gateway, one NAT gateway, route tables, IAM roles, EKS 1.37 with one `t3.small` node group, two ECR repositories and an encrypted S3 bucket. 33 resources, all tagged `Project=incident-tracker`.

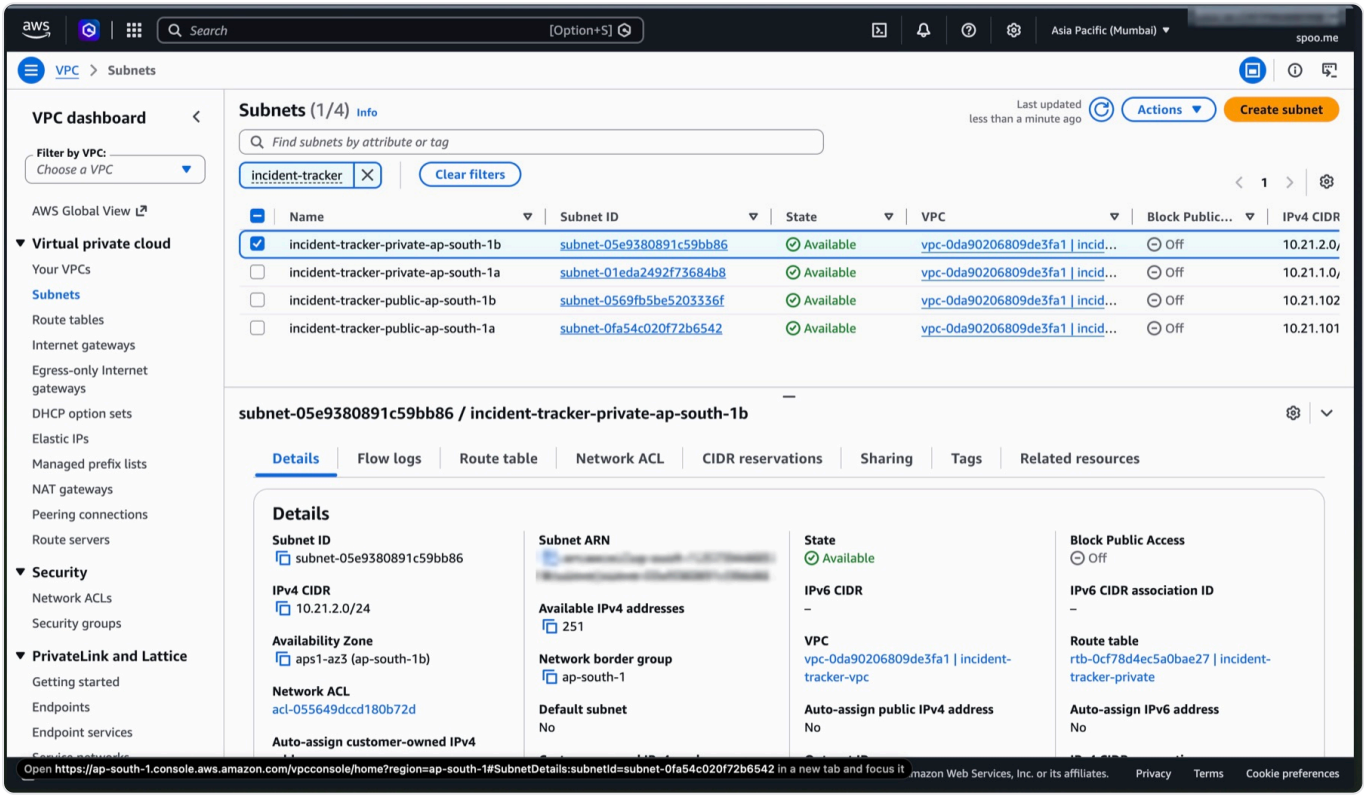
CRITERION	PTS	EVIDENCE
`terraform/` with valid HCL	2	`main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`. `terraform validate` passes.
`terraform init` succeeds	1	Screenshot 14.
`terraform plan` is non-empty with no errors	2	`Plan: 33 to add, 0 to change, 0 to destroy.`
VPC with at least two public subnets	3	`incident-tracker-public-ap-south-1a` and `-1b`, plus two private subnets. Screenshots 18 and 19.
EKS cluster with a node group	4	`incident-tracker-eks` Active, node group `incident-tracker-default`. Screenshot 20.
`terraform destroy` cleans up	2	33 destroyed, state empty, and direct AWS queries return 0 clusters, VPCs, NAT gateways, Elastic IPs and buckets. Screenshot 05.
`terraform.tfvars.example`, no credentials	1	Present. Credentials come from the AWS CLI chain. gitleaks scans the full history on every run with 0 findings.

```
$ terraform init -input=false -no-color | grep -E 'Installed|successfully initialized'
Terraform has been successfully initialized!
$ terraform plan -input=false -no-color -out=tfplan | grep -E '^  # |^Plan:'
  # aws_cloudwatch_log_group.eks will be created
  # aws_ecr_repository.app["backend"] will be created
  # aws_ecr_repository.app["frontend"] will be created
  # aws_eip.nat will be created
  # aws_eks_cluster.main will be created
  # aws_eks_node_group.main will be created
  # aws_iam_role.cluster will be created
  # aws_iam_role.nodes will be created
  # aws_iam_role_policy_attachment.cluster will be created
  # aws_iam_role_policy_attachment.nodes["AmazonEC2ContainerRegistryReadOnly"] will be created
  # aws_iam_role_policy_attachment.nodes["AmazonEKSWorkerNodePolicy"] will be created
  # aws_iam_role_policy_attachment.nodes["AmazonEKS_CNI_Policy"] will be created
  # aws_internet_gateway.main will be created
  # aws_nat_gateway.main will be created
  # aws_route_table.private will be created
  # aws_route_table.public will be created
  # aws_route_table_association.private["ap-south-1a"] will be created
  # aws_route_table_association.private["ap-south-1b"] will be created
  # aws_route_table_association.public["ap-south-1a"] will be created
  # aws_route_table_association.public["ap-south-1b"] will be created
  # aws_s3_bucket.backups will be created
  # aws_s3_bucket_public_access_block.backups will be created
  # aws_s3_bucket_server_side_encryption_configuration.backups will be created
  # aws_s3_bucket_versioning.backups will be created
  # aws_security_group.cluster will be created
  # aws_subnet.private["ap-south-1a"] will be created
  # aws_subnet.private["ap-south-1b"] will be created
  # aws_subnet.public["ap-south-1a"] will be created
  # aws_subnet.public["ap-south-1b"] will be created
  # aws_vpc.main will be created
  # aws_vpc_security_group_egress_rule.cluster_all will be created
  # aws_vpc_security_group_ingress_rule.api_from_vpc will be created
  # random_id.bucket will be created
Plan: 33 to add, 0 to change, 0 to destroy.
$
```

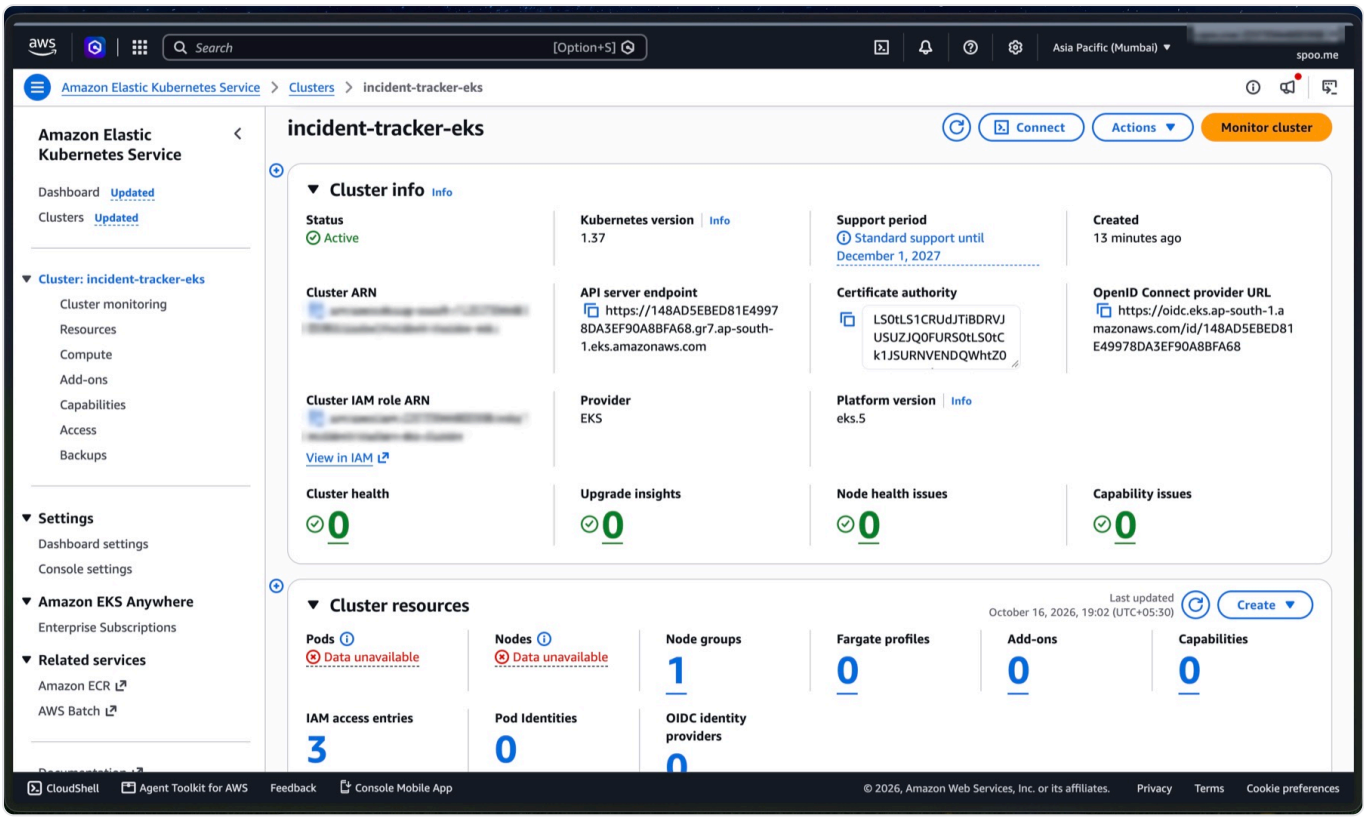
14 · terraform init and plan



18 · the VPC in the AWS Console, Asia Pacific (Mumbai). Account number blurred.



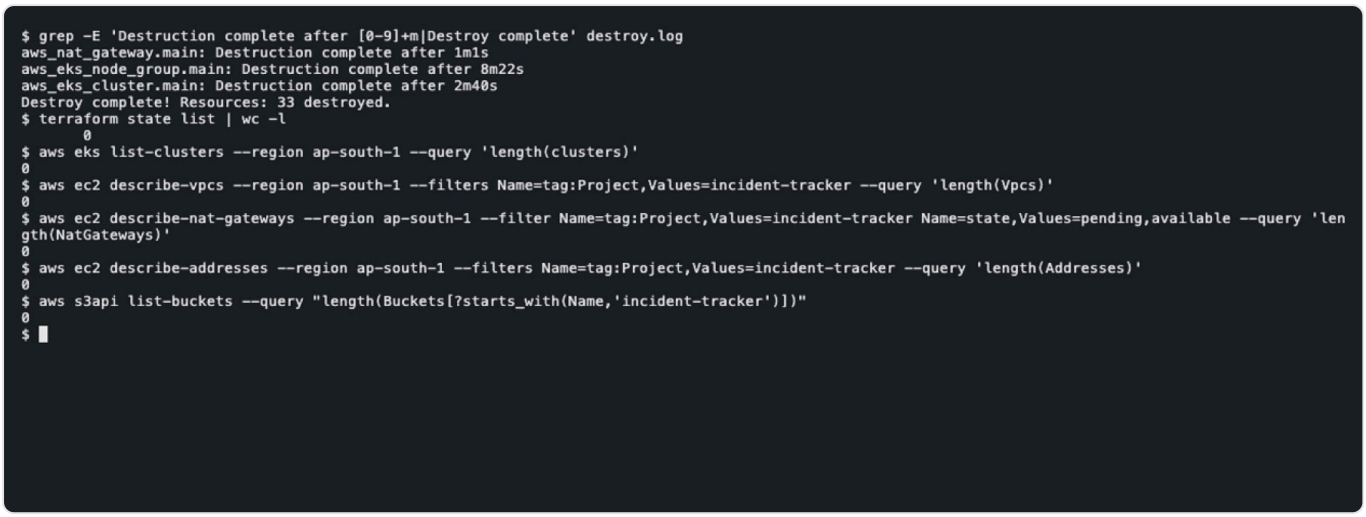
19 · two public and two private subnets



20 · the EKS cluster Active with one node group

```
$ grep -E 'Creation complete after [0-9]+m|Apply complete|^real' apply.log
aws_nat_gateway.main: Creation complete after 1m24s [id=nat-006a1c09b6cf9baf4]
aws_eks_cluster.main: Creation complete after 8m14s [id=incident-tracker-eks]
aws_eks_node_group.main: Creation complete after 1m52s [id=incident-tracker-eks:incident-tracker-default]
Apply complete! Resources: 33 added, 0 changed, 0 destroyed.
$ terraform output -no-color | grep -vE 'subnet-|^]|_ids|kubeconfig' | sed -E 's/[0-9]{12}[.]dkr/<account-id>.dkr/'
backup_bucket = "incident-tracker-backups-4a3526"
ecr_repository_urls = {
  "backend" = "<account-id>.dkr.ecr.ap-south-1.amazonaws.com/incident-tracker/backend"
  "frontend" = "<account-id>.dkr.ecr.ap-south-1.amazonaws.com/incident-tracker/frontend"
}
eks_cluster_endpoint = "https://6F68A598A2777766E2D95AD053DFBE79.gr7.ap-south-1.eks.amazonaws.com"
eks_cluster_name = "incident-tracker-eks"
eks_cluster_status = "ACTIVE"
eks_log_group = "/aws/eks/incident-tracker-eks/cluster"
node_group_status = "ACTIVE"
vpc_id = "vpc-0d6ea33f38886a841"
$ kubectl get nodes -o wide
NAME                                          STATUS   ROLES    AGE   VERSION               INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
 KERNEL-VERSION                       CONTAINER-RUNTIME
ip-10-21-1-143.ap-south-1.compute.internal   Ready    <none>   16m   v1.37.0-eks-3b4a6ca   10.21.1.143   <none>        Amazon Linux 2023.12.20260928
 6.18.51-120.162.amzn2023.x86_64 (amd64)   containerd://2.2.7+unknown
$ kubectl get pods -o wide; curl -s localhost:18225/ | grep -o '<title>.*</title>'
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE                                          NOMINATED NODE   RE
ADINESS GATES
incidents-frontend-55f8674d75-z7ztb   1/1     Running   0          13m   10.21.1.69   ip-10-21-1-143.ap-south-1.compute.internal   <none>           <n
one>
<title>Incident Tracker</title>
$
```

04 · an earlier apply, with the EKS node Ready and a pod running on it



05 · terraform destroy, then every AWS check returns 0

M8 · Kubernetes and Helm

15 pts

The chart deploys the backend and frontend Deployments, Postgres as a StatefulSet with a PVC, a ConfigMap, a Secret, ClusterIP Services, an Ingress, an HPA, a ServiceMonitor, a Grafana dashboard and a `helm test` smoke pod. The image tag is required, so a deploy always names an exact build:

```
kubectl apply -f k8s/namespace.yaml
helm upgrade --install incidents helm/incidents -n incidents --set backend.tag=<sha> --set frontend.tag=<sh
a>
```

CRITERION	PTS	EVIDENCE
`k8s/namespace.yaml` applies	1	Namespace `incidents`, applied in CI and locally.
Helm chart with `Chart.yaml`, `values.yaml`, templates	3	`helm/incidents/`. `helm lint` passes with the image tags set.
`helm upgrade --install` deploys	3	Runs in every CI deploy with `--wait`, release status `deployed`.
Backend and frontend with at least 2 replicas	2	Frontend 2 replicas. Backend held at 2 to 5 by the HPA (screenshot 03 was taken under load at 5).
ClusterIP Services	2	`incidents-backend`, `incidents-frontend`, `incidents-postgres`, all ClusterIP.
Ingress sends `/` to frontend, `/api` to backend	2	`templates/ingress.yaml` on host `incidents.localhost`, ingress-nginx.
All pods Running	2	Screenshot 03.

```
$ helm list -n incidents
NAME            NAMESPACE       REVISION        UPDATED                                 STATUS          CHART           APP VERSION
incidents       incidents       2               2026-10-07 21:09:15.239661 +0530 IST    deployed        incidents-1.0.0 1.0.0
$ kubectl -n incidents get pods,svc,ingress,hpa,pvc
NAME                                 READY   STATUS    RESTARTS   AGE
pod/incidents-backend-7cbdb9f8bb-2tv6w   1/1     Running   0          6m
pod/incidents-backend-7cbdb9f8bb-5qcvt   1/1     Running   0          6m15s
pod/incidents-backend-7cbdb9f8bb-8qwcb   1/1     Running   0          12m
pod/incidents-backend-7cbdb9f8bb-qrtgb   1/1     Running   0          12m
pod/incidents-backend-7cbdb9f8bb-spvqc   1/1     Running   0          6m15s
pod/incidents-frontend-6df95d7-bbwjn     1/1     Running   0          12m
pod/incidents-frontend-6df95d7-lhd7c     1/1     Running   0          12m
pod/incidents-postgres-0                 1/1     Running   0          12m

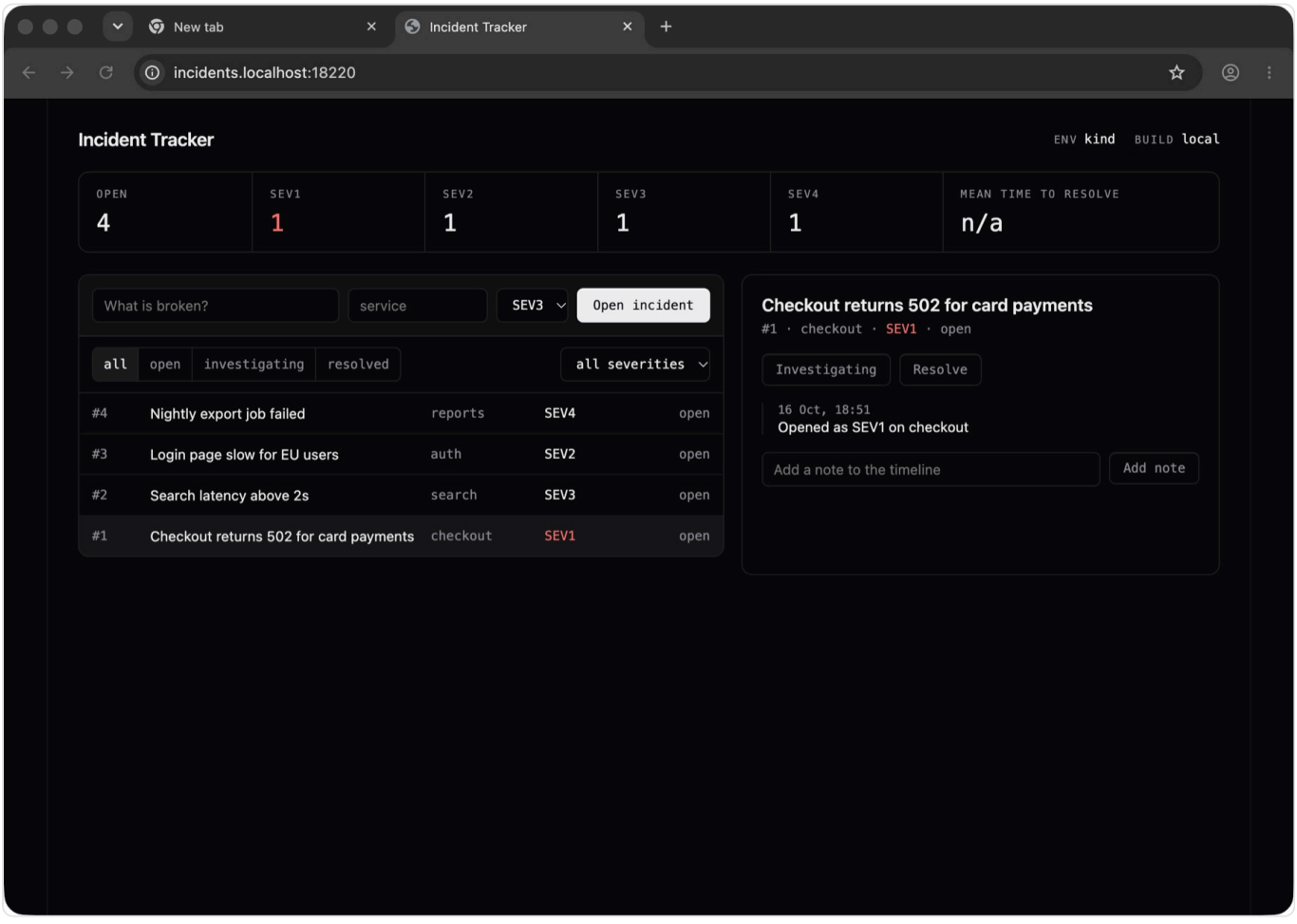
NAME                         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/incidents-backend    ClusterIP   10.96.81.72     <none>        8000/TCP   12m
service/incidents-frontend   ClusterIP   10.96.182.132   <none>        80/TCP     12m
service/incidents-postgres   ClusterIP   10.96.109.169   <none>        5432/TCP   12m

NAME                                CLASS   HOSTS                ADDRESS     PORTS   AGE
ingress.networking.k8s.io/incidents   nginx   incidents.localhost   localhost   80      12m

NAME                                                          REFERENCE                       TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/incidents-backend   Deployment/incidents-backend   cpu: 37%/70%   2         5         5          12m

NAME                                          STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEA
TTRIBUTESCLASS   AGE
persistentvolumeclaim/data-incidents-postgres-0   Bound    pvc-ebaafae4-1224-431a-a02e-d11783073d03   1Gi        RWO            standard       <unset>
                 12m
$ kubectl -n incidents get cm incidents-config -o jsonpath='{.data}'; echo; kubectl -n incidents get secret incidents-db -o jsonpath='{.data}' | jq -c
 'keys'
{"APP_VERSION":"local","ENVIRONMENT":"kind","SITE_TITLE":"Incident Tracker"}
["DATABASE_URL","POSTGRES_PASSWORD","POSTGRES_USER"]
$ curl -s http://incidents.localhost:18220/api/stats; echo; kubectl -n incidents exec deploy/incidents-backend -c backend -- python -c "import urllib.
request as u; print(u.urlopen('http://localhost:8000/metrics').read().decode())" | grep -E '^incidents_(created_total|resolved_total|open|time_to_reso
lve_seconds_count)'
{"open":926,"open_by_severity":{"SEV1":237,"SEV2":239,"SEV3":225,"SEV4":225},"resolved":1998,"mean_time_to_resolve_seconds":0.2694323518518518}
incidents_created_total{severity="SEV1"} 165.0
incidents_created_total{severity="SEV2"} 202.0
incidents_created_total{severity="SEV4"} 197.0
incidents_created_total{severity="SEV3"} 208.0
incidents_resolved_total 534.0
incidents_time_to_resolve_seconds_count 534.0
incidents_open 926.0
$
```

03 · helm list, kubectl get pods, svc, ingress, hpa and pvc



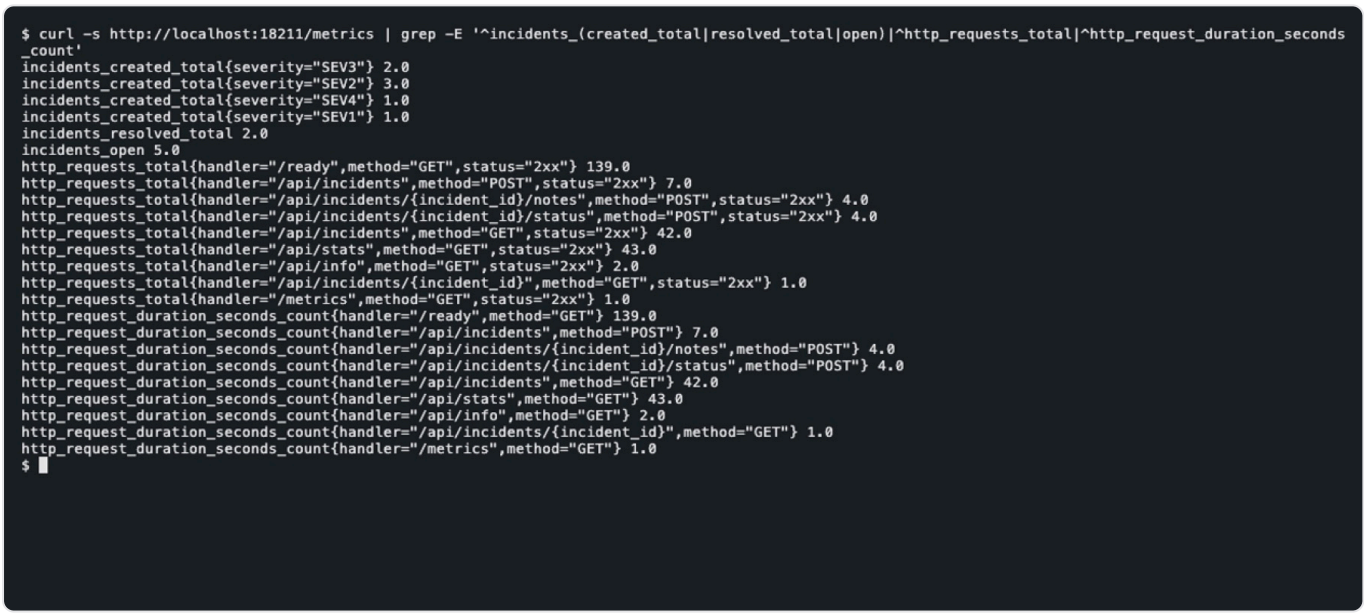
01 · the app through the Ingress at incidents.localhost

M9 · Observability

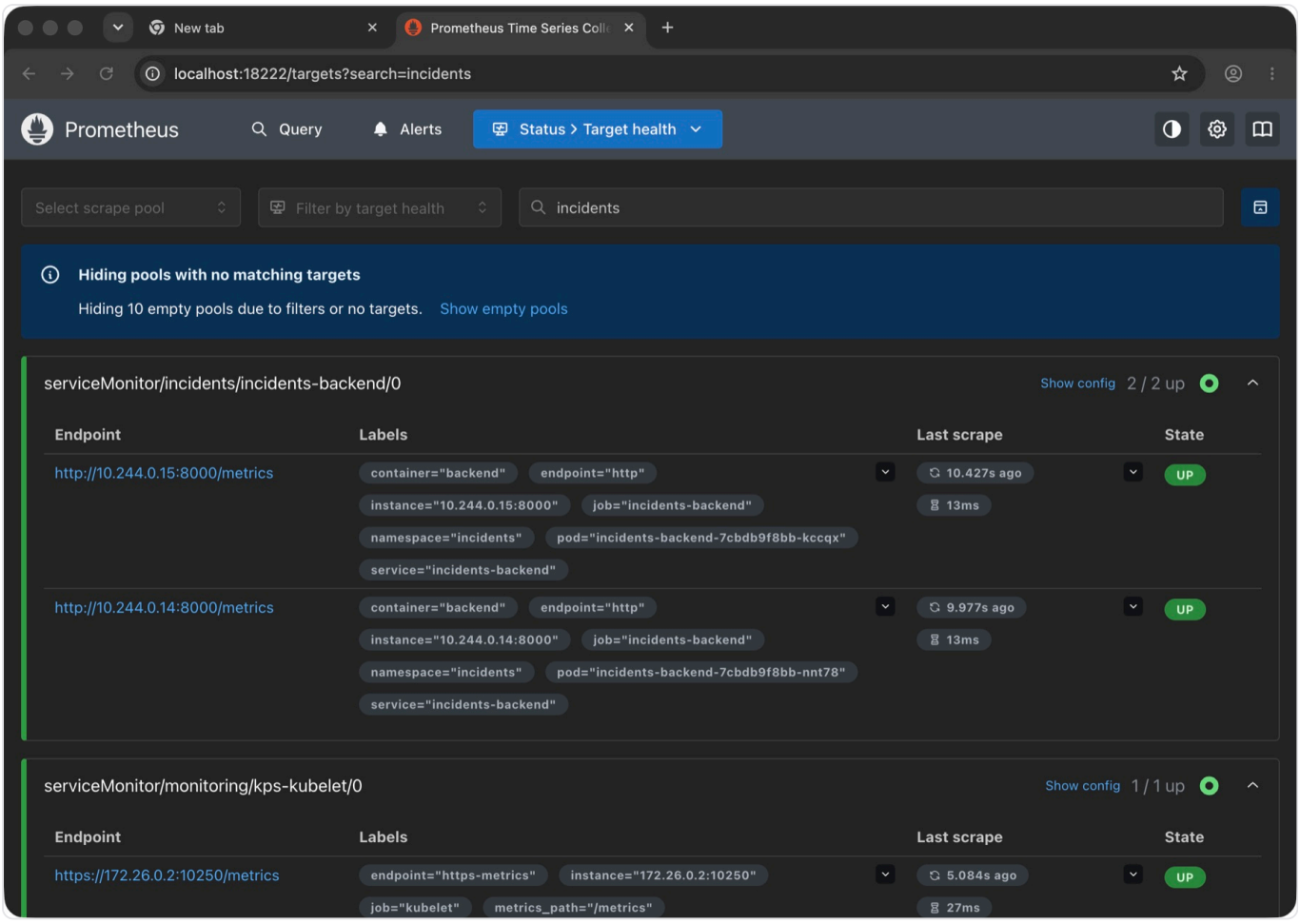
10 pts

The backend exposes HTTP metrics from `prometheus-fastapi-instrumentator` and app metrics of its own: incidents opened by severity, incidents resolved, time to resolve, and open incidents read from the database at scrape time. kube-prometheus-stack runs in the cluster with `monitoring/kube-prometheus-stack-values.yaml`.

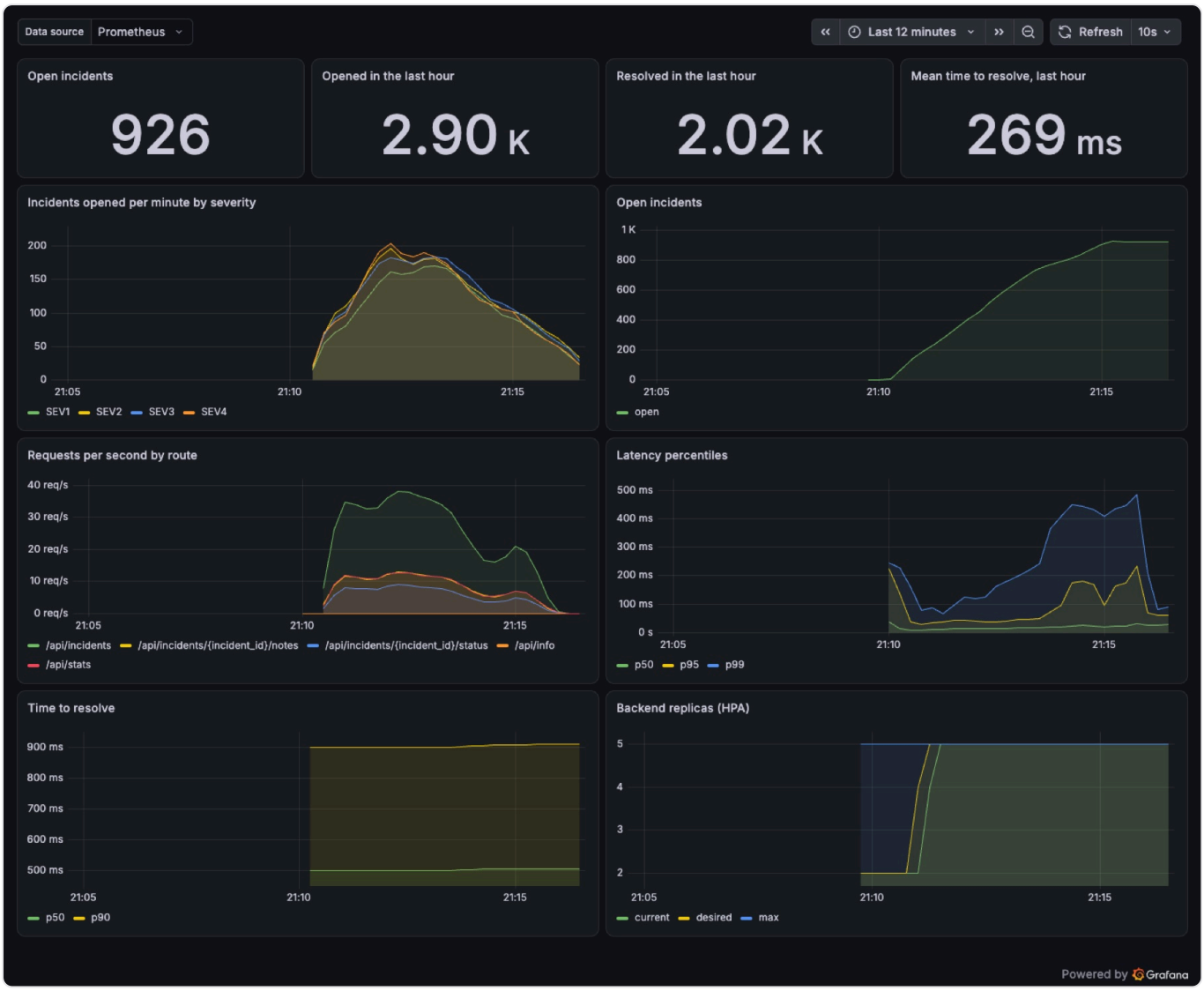
CRITERION	PTS	EVIDENCE
`/metrics` returns Prometheus output	2	Screenshot 13.
Prometheus scrapes the app	3	The chart's ServiceMonitor. Both backend pods are UP on the Targets page.
Grafana installed and reachable	2	Part of kube-prometheus-stack. The dashboard ships as a ConfigMap that Grafana's sidecar loads.
A panel with live app metrics	3	Request rate, latency, incidents by severity, open incidents, time to resolve, HPA replicas.



13 · curl /metrics on the backend



17 · Prometheus Targets, both backend pods UP



08 · the incidents dashboard in Grafana under load

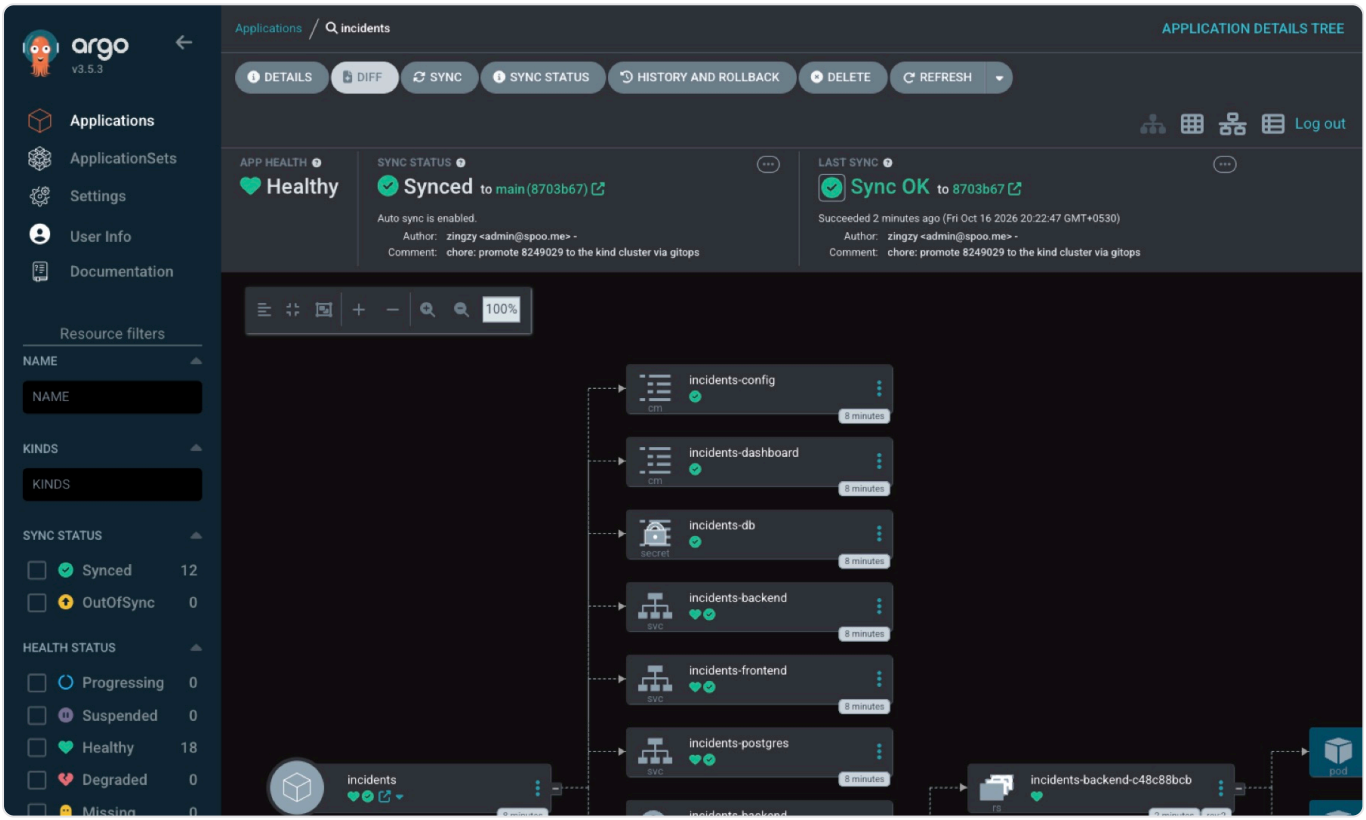
M10 · README and demo

5 pts

The README in the repository root covers the overview, architecture, setup, Docker, Kubernetes, Helm, Terraform, CI/CD, DevSecOps, monitoring, GitOps, seven failure drills, screenshots and lessons learned.

The demo flow: commit a change and push to `main`, the pipeline tests, scans and pushes images tagged with that SHA, then a one-line promotion commit in `gitops/incidents-kind/values.yaml` moves Argo CD to the new build. The last real promotion, polled every 10 seconds:

```
20:20:58 pushed 8703b67, image tags local -> 82490290...
20:21:02 argo=dffd3e9 Synced/Healthy     version=local
20:22:53 argo=8703b67 Synced/Progressing version=local
20:23:24 argo=8703b67 Synced/Progressing version=82490290033c676d9d492f9c227429dfbaf532fa
20:23:54 argo=8703b67 Synced/Healthy     version=82490290033c676d9d492f9c227429dfbaf532fa
```



09 · Argo CD Synced and Healthy at 8703b67